

REVISÃO

1. O QUE É O PHP?

O PHP (Hypertext Preprocessor) é uma linguagem de programação focada no **lado do servidor (Back-end)**.

Pense na web como um restaurante:

- O **HTML/CSS (Front-end)** é o cardápio e a decoração do salão - é o que o cliente (usuário) vê e interage.
- O **PHP (Back-end)** é a cozinha. Quando o cliente faz um pedido (clica em "enviar" num formulário), o garçom (a internet) leva o pedido para a cozinha. O php processa os ingredientes (dados), consulta o estoque (banco de dados) e devolve um prato pronto (uma página html pura) para o cliente. O cliente nunca vê como a comida foi feita, apenas o resultado.

2. A SINTAXE BÁSICA E AS TAGS FUNDAMENTAIS

Para que o servidor saiba que um trecho de código é PHP e não um simples texto HTML, precisamos usar as tags de abertura e fechamento.

- **Abertura e Fechamento:** Tudo que é PHP deve estar dentro de `<?php ?>`.
- **A Tag de Saída (echo):** É o comando mais básico. Ele serve para "imprimir" (mostrar) algo na tela.

Ex:

```
<?php  
    echo "Olá, mundo!";  
?>
```

- **Ponto e Vírgula (;):** Quase toda instrução em PHP **precisa** terminar com ponto e vírgula. É como o ponto final de uma frase. Esquecer isso é a causa número um de telas de erro.

3. VARIÁVEIS E TIPOS DE DADOS

Como vimos na revisão anterior, variáveis são "caixas" onde guardamos informações.

- **Regra de ouro:** toda variável em php deve começar com o cifrão \$.
- **Tipagem dinâmica:** o php descobre o tipo de dado sozinho.

Ex:

```
<?php  
  
$nome = "Maria"; // O PHP sabe que é texto (String)  
  
$idade = 25;    // O PHP sabe que é um número inteiro (Integer)  
  
$preco = 19.99; // O PHP sabe que é um número decimal (Float)  
  
?>
```

4. ARRAYS (VETORES) – ALOCAÇÃO DE MEMÓRIA

Arrays são variáveis especiais que conseguem guardar vários valores ao mesmo tempo, como se fosse um gaveteiro.

- **Array indexado:** organizado por números (começando sempre do zero).

```
$frutas = ["Maçã", "Banana", "Laranja"];
```

```
echo $frutas[0]; // Vai imprimir "Maçã"
```

```
echo $frutas[1]; // Vai imprimir "Banana"
```

```
echo $frutas[2]; // Vai imprimir "Laranja"
```

Array associativo: em vez de números, usamos "etiquetas" (palavras) para organizar as gavetas. é exatamente o formato que o banco de dados devolve quando usamos o `fetch_assoc`.

Ex:

```
$usuario = [
```

```
    "nome" => "Maria",
```

```
    "email" => "mcandida@gmail.com"
```

```
];
```

```
echo $usuario["nome"]; // Vai imprimir "Maria"
```

5. ESTRUTURAS DE CONTROLE (DECISÃO E REPETIÇÃO)

Essas tags controlam o "cérebro" do código, decidindo o que fazer e quando fazer.

- **O if e else (se / senão):** Toma decisões com base em condições.

```
$nota = 7;

if ($nota >= 6) {

    echo "Aprovado!";

} else {

    echo "Reprovado.";

}
```

O foreach (para cada): é um laço de repetição criado especificamente para ler arrays. Ele passa por cada "gaveta" automaticamente. Foi o que usamos para criar a tabela de listagem do banco de dados.

```
$alunos = ["Ana", "Carlos", "Beatriz"];

foreach ($alunos as $aluno) {

    echo $aluno . "<br>";

}
```

6. VARIÁVEIS SUPERGLOBAIS

O PHP tem variáveis nativas, escritas sempre em MAIÚSCULAS e que estão sempre disponíveis para pegar informações soltas no servidor. As mais importantes para comunicação web são:

- **\$_POST:** Captura dados enviados por um formulário de forma oculta (segura). Ideal para senhas, textos longos ou cadastro no banco de dados.
- **\$_GET:** Captura dados que são enviados visivelmente através da URL (ex: `site.com/perfil.php?id=10`). É ideal para criar links de edição ou exclusão, mas nunca para enviar senhas.
- **\$_SESSION:** Guarda informações do usuário enquanto ele navega pelo site (como o login ou itens de um carrinho de compras). O servidor lembra de quem é aquele navegador até que ele seja fechado.

7. VARIÁVEIS EM PHP

Em PHP, as variáveis são os "recipientes" onde guardamos os dados que transitam entre o formulário HTML e o banco de dados.

- **Declaração:** Toda variável começa obrigatoriamente com o cifrão \$. Exemplo: `$conexao`, `$id`, `$nome`.
- **Tipagem dinâmica:** você não precisa avisar ao PHP se a variável vai guardar um texto ou um número; ele descobre sozinho quando você atribui o valor.
- **Superglobais (arrays especiais):** Nos seus códigos, aparecem muito o `$_POST` e o `$_GET`. Eles são arrays criados automaticamente pelo PHP.
 - `$_POST` captura os dados escondidos enviados por um formulário (usado no seu `atualizar.php`).

- `$_GET` captura dados visíveis passados pela URL, como o ID para exclusão (ex: `editar.php?id=5`).
- Estou praticamente repetindo o que eu falei em variáveis `SUPERGLOBAIS` mas com uma explicação um pouco mais aprimorada (suponho, maybe).

8. ANÁLISE LINHA POR LINHA: AS TAGS E FUNÇÕES CRÍTICAS

A REDE DE SEGURANÇA: TRY E CATCH

- **try { ... }:** Significa "tente executar este bloco de código". É usado para envolver operações arriscadas, como conectar ao banco de dados ou fazer uma consulta complexa. Se tudo der certo, o PHP ignora o catch.
- **catch (PDOException \$e) { ... }:** Se o banco de dados cair ou a senha estiver errada, o código dentro do try "explode". O catch "captura" essa explosão (a exceção) para que a tela não exiba um erro feio expondo o servidor. O `$e` guarda os detalhes do erro.

O BOTÃO DE PÂNICO: DIE() E EXIT()

- **die("Erro..."):** Literalmente mata a execução do script naquela linha. No nosso código, se a conexão falhar, o `die()` exibe a mensagem de erro e impede que o PHP tente buscar dados num banco ao qual não conseguiu se conectar.
- **exit():** Funciona de forma idêntica ao `die`. Em um dos nossos códigos, após o `header("Location: listar.php")` mandar o usuário de volta para a tabela, usamos o `exit()` para garantir que o servidor pare de ler qualquer código abaixo daquela linha.

O ESCUDO CONTRA CRACKERS: PREPARE(), BINDPARAM() E EXECUTE()

(ESCUDO NÃO SIGNIFICA IMPENETRÁVEL E NEM IMPEDIMENTO, SE O CRACKER ESTIVER DISPOSTO A INVADIR, ELE VAI FICAR HORAS E HORAS TENTANDO ATÉ CONSEGUIR)

Essas três funções trabalham juntas para evitar o SQL Injection (quando usuários mal-intencionados digitam comandos no formulário para apagar seu banco).

- **prepare(\$sql)**: Em vez de colocar as variáveis direto na frase SQL, criamos um molde com espaços vazios (:nome, :email). O prepare avisa ao banco: "Vou te mandar um comando, prepare-se para receber as peças".
- **bindParam(':nome', \$nome)**: É a cola. Ele liga o espaço vazio :nome à sua variável PHP \$nome de forma 100% segura, tratando tudo como texto inofensivo.
- **execute()**: Dá a ordem final. Pega o molde preparado, encaixa as variáveis "coladas" e roda no banco de dados.

O TRADUTOR: FETCHALL(PDO::FETCH_ASSOC)

- Essa função pega a resposta "crua" do banco de dados e a transforma em um Array Associativo que o PHP consegue ler facilmente. É aqui que entra a regra de ouro da Aula 15: as chaves desse array são **estritamente case-sensitive**. Se a coluna no banco chama NOME, a variável tem que ser acessada como \$linha['NOME']. Eu expliquei isso no PDF, inclusive. E como eu aprendi isso na prática? Errando kkk por isso tem a explicação do erro no PDF de vocês, pra vocês se encontrarem.

Referências: várias mas to sem tempo pra colocar direitinho no pdf, sorry <3